國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 05

姓名：黃聖傑

學號：114310140

Lab05-Q1&Q2. (題目編號)
Q1:Complete the following parts in the sample code: **- Class Stack with 'push', 'pop', and 'isEmpty',shuffleDeck,drawALLCards** **Q2:Add comments to the program**

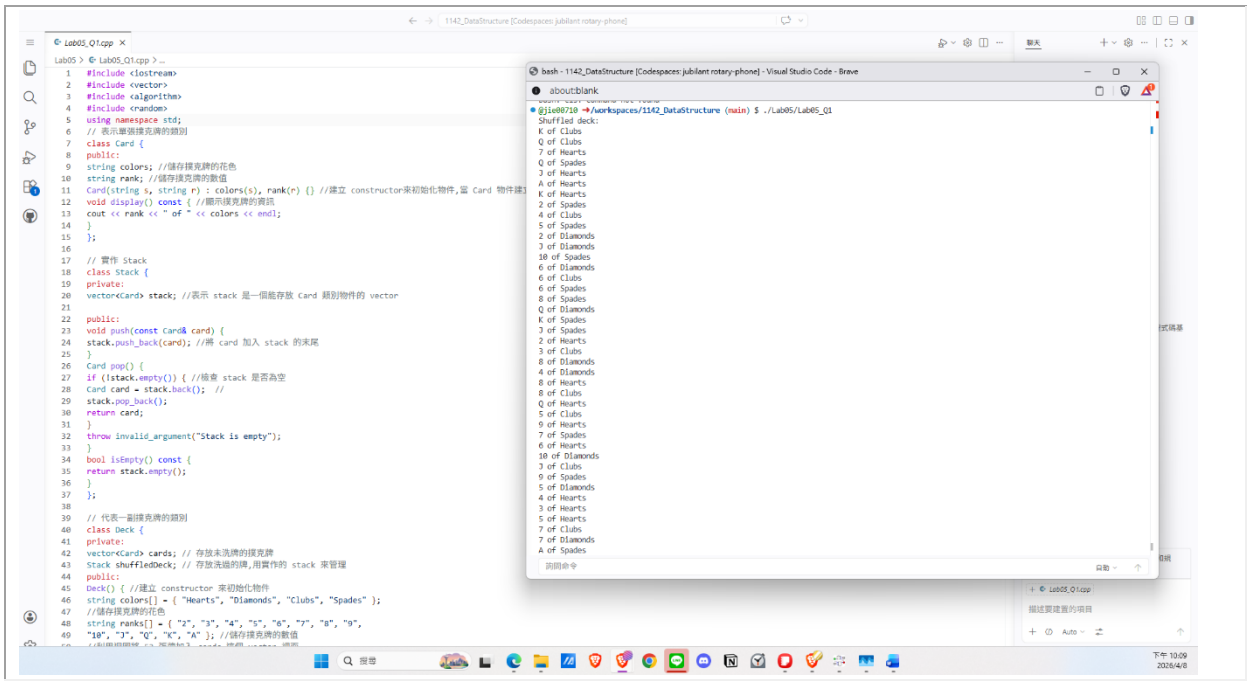
Code

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>
using namespace std;
// 表示單張撲克牌的類別
class Card {
public:
string colors; //儲存撲克牌的花色
string rank; //儲存撲克牌的數值
Card(string s, string r) : colors(s), rank(r) {} //建立 constructor 來初始化物件,
當 Card 物件建立時,它會自動執行這個函式,並把 s 和 r 的值存入 colors 和 rank
void display() const { //顯示撲克牌的資訊
cout << rank << " of " << colors << endl;
}
};

// 實作 Stack
class Stack {
private:
vector<Card> stack; //表示 stack 是一個能存放 Card 類別物件的 vector

public:
void push(const Card& card) { //push 函式
stack.push_back(card); //將 card 加入 stack 的末尾
}
Card pop() { //pop 函式
if (!stack.empty()) { //檢查 stack 是否為空，若不為空則執行下列程式
        Card card = stack.back();  //暫存 stack 中最頂端的元素
        stack.pop_back(); //使用 vector 內的 pop 函式移除該元素並將 top-1
        return card; //回傳剛剛暫存的元素
}
```

```
throw invalid_argument("Stack is empty");//若stack為空，則丟出stack is empty訊息，並跳出副程式
}
bool isEmpty() const { //如果讀取到的值是 isEmpty = True，則回傳stack.emoty()
return stack.empty();
}
};

// 代表一副撲克牌的類別
class Deck {
private:
vector<Card> cards; // 存放未洗牌的撲克牌
Stack shuffledDeck; // 存放洗過的牌,用實作的 stack 來管理
public:
Deck() { //建立 constructor 來初始化物件
string colors[] = { "Hearts", "Diamonds", "Clubs", "Spades" };
//儲存撲克牌的花色
string ranks[] = { "2", "3", "4", "5", "6", "7", "8", "9",
"10", "J", "Q", "K", "A" }; //儲存撲克牌的數值
//利用迴圈將 52 張牌加入 cards 這個 vector 裡面
for (int i = 0; i < 4; i++) { // 四種花色
for (int j = 0; j < 13; j++) { // 13 種點數
cards.push_back(Card(colors[i], ranks[j]));
}
}
}
//洗牌(Hint:使用 shuffle 函數)
void shuffleDeck() {
        random_device rd; //確保順序隨機
        mt19937 g(rd()); //隨機數產生器
        shuffle(cards.begin(), cards.end(), g); //接收card的起點與終點，並打亂順序

        for (int i = 0; i < cards.size(); i++) { //將排重新放回stack
            shuffledDeck.push(cards[i]);
        }
}
//發牌
```

```
void drawAllCards() {
    while (!shuffledDeck.isEmpty()) { //判斷 shuffledDeck 是否還有值,如果不是空的就重複執行迴圈
    Card card = shuffledDeck.pop(); //從 shuffledDeck 中取出一張牌
        card.display(); //顯示這張牌的資訊
    }
}
};

int main() {
    Deck deck; //建立 deck 產生 52 張撲克牌
    deck.shuffleDeck(); //進行洗牌並放入堆疊
    cout << "Shuffled deck:" << endl;
    deck.drawAllCards(); //依序取出堆疊內的牌並顯示
    return 0; //回傳 0，表示程式正常結束
```

Result



Lab05_Q1.cpp

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>
using namespace std;
// 表示單張撲克牌的類別
class Card {
public:
string colors; //儲存撲克牌的花色
string rank; //儲存撲克牌的數值
Card(string s, string r) : colors(s), rank(r) {} //建立 constructor來初始化物件,當 Card 物件建
void display() const { //顯示撲克牌的資訊
cout << rank << " of " << colors << endl;
}
};

// 實作 Stack
class Stack {
private:
vector<Card> stack; //表示 stack 是一個能存放 Card 類別物件的 vector

public:
void push(const Card& card) {
stack.push_back(card); //將 card 加入 stack 的末尾
}
Card pop() {
if (!stack.empty()) { //檢查 stack 是否為空
Card card = stack.back();  //
stack.pop_back();
return card;
}
throw invalid_argument("Stack is empty");
}
bool isEmpty() const {
return stack.empty();
}
};

// 代表一副撲克牌的類別
class Deck {
private:
vector<Card> cards; // 存放未洗牌的撲克牌
Stack shuffledDeck; // 存放洗過的牌,用實作的 stack 來管理
public:
Deck() { //建立 constructor 來初始化物件
string colors[] = { "Hearts", "Diamonds", "Clubs", "Spades" };
//儲存撲克牌的花色
string ranks[] = { "2", "3", "4", "5", "6", "7", "8", "9",
"10", "J", "Q", "K", "A" }; //儲存撲克牌的數值
```

bash - 1142_DataStructure [Codespaces: jubilant rotary-phone] - Visual Studio Code - Brave

```
@jie00710 ➜ /workspaces/1142_DataStructure (main) $ ./Lab05/Lab05_Q1
Shuffled deck:
4 of Spades
10 of Spades
J of Hearts
J of Clubs
K of Hearts
5 of Diamonds
Q of Spades
6 of Clubs
Q of Diamonds
10 of Clubs
5 of Spades
3 of Hearts
7 of Spades
7 of Clubs
9 of Clubs
A of Clubs
7 of Diamonds
6 of Diamonds
8 of Diamonds
A of Hearts
9 of Spades
2 of Spades
4 of Clubs
3 of Spades
6 of Spades
2 of Diamonds
8 of Clubs
5 of Clubs
Q of Hearts
8 of Spades
A of Diamonds
10 of Diamonds
7 of Hearts
K of Diamonds
10 of Hearts
4 of Hearts
J of Diamonds
A of Spades
4 of Diamonds
6 of Hearts
K of Clubs
```

Lab05_Q1.cpp

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>
using namespace std;
// 表示單張撲克牌的類別
class Card {
public:
string colors; //儲存撲克牌的花色
string rank; //儲存撲克牌的數值
Card(string s, string r) : colors(s), rank(r) {} //建立 constructor來初始化物件,當 Card 物件建
void display() const { //顯示撲克牌的資訊
cout << rank << " of " << colors << endl;
}
};

// 實作 Stack
class Stack {
private:
vector<Card> stack; //表示 stack 是一個能存放 Card 類別物件的 vector

public:
void push(const Card& card) {
stack.push_back(card); //將 card 加入 stack 的末尾
}
Card pop() {
if (!stack.empty()) { //檢查 stack 是否為空
Card card = stack.back();  //
stack.pop_back();
return card;
}
throw invalid_argument("Stack is empty");
}
bool isEmpty() const {
return stack.empty();
}
};

// 代表一副撲克牌的類別
class Deck {
private:
vector<Card> cards; // 存放未洗牌的撲克牌
Stack shuffledDeck; // 存放洗過的牌,用實作的 stack 來管理
public:
Deck() { //建立 constructor 來初始化物件
string colors[] = { "Hearts", "Diamonds", "Clubs", "Spades" };
//儲存撲克牌的花色
string ranks[] = { "2", "3", "4", "5", "6", "7", "8", "9",
"10", "J", "Q", "K", "A" }; //儲存撲克牌的數值
```

bash - 1142_DataStructure [Codespaces: jubilant rotary-phone] - Visual Studio Code - Brave

```
@jie00710 ➜ /workspaces/1142_DataStructure (main) $ ./Lab05/Lab05_Q1
Shuffled deck:
Q of Clubs
9 of Clubs
9 of Diamonds
6 of Clubs
J of Diamonds
5 of Hearts
8 of Diamonds
8 of Spades
6 of Diamonds
3 of Diamonds
9 of Hearts
4 of Diamonds
8 of Clubs
6 of Hearts
5 of Diamonds
6 of Spades
8 of Hearts
A of Hearts
Q of Spades
7 of Spades
4 of Spades
5 of Spades
3 of Hearts
4 of Hearts
4 of Clubs
3 of Clubs
A of Spades
9 of Spades
2 of Clubs
Q of Hearts
10 of Hearts
2 of Diamonds
J of Hearts
3 of Spades
Q of Diamonds
7 of Hearts
10 of Clubs
J of Clubs
A of Diamonds
K of Clubs
10 of Diamonds
```